

WAB

Seaborn und Matplotlib im Vergleich
Benutzerfreundlichkeit und Funktionsumfang

Zur Veranstaltung Programmierung
im Studiengang ‚Informatik‘

vorgelegt dem Fachbereich
Informatik und Wirtschaftsinformatik
der
Provadis School of International Management and Technology
von

Jeremy Blum
D960
Informatik BIN-T24-F3
jeremy.blum@stud-provadis-hochschule.de

Erstgutachter: Prof. Dr. Henrik Paul

Ende der Bearbeitungsfrist: 14.04.2025

Steckbrief zur Erstellung einer Studienarbeit

Ersteller:	Name		Vorname		E-Mailadresse		
Information-Owner Deutsche Telekom: (für die Informationen verantwortlich)	Gesellschaft		OrgE				
	Name		Vorname		E-Mailadresse		
Institut/Hochschule:	Name		Land/Stadt		Homepage		
	Betreuer der Arbeit:	Name		Vorname		E-Mailadresse	
Titel der Arbeit:							
Kurzbeschreibung:							
Für die Studienarbeit ermittelte Schutzklasse:		"OFFEN"		"INTERN"		"VERTRAULICH"	
		d.h. unterstelltes Unternehmensinteresse*		d.h. nachgewiesenes Unternehmensinteresse*			
Hinweis zu "VERTRAULICH": Studienarbeiten der Schutzklasse VERTRAULICH-TBS dürfen nicht erstellt werden. Die Schutzklasse VERTRAULICH-TBS liegt vor, wenn ein Top Business Secret in der Arbeit enthalten ist. Diese Informationen werden auf Ebene des Vorstand bzw. der Geschäftsführung, z. B. durch den Stabsbereich, festgelegt und alle Mitarbeiter die damit umgehen wissen, dass es sich dabei um ein Top Business Secret handelt. Ein TBS wird nach einem individuellen Schutzkonzept geschützt.							
Hilfsmittel zur Ermittlung der Schutzklasse: Informationsdrehscheibe				* Das Unternehmensinteresse wird jeweils mit Freigabe durch den Information-Owner bestätigt.			
Die unten angegebenen Dokumente/ Informationen wurden für die Erstellung der Arbeit genutzt und begründen die Schutzklasse.							
Für die Studienarbeit konkret festgelegte Schutzmaßnahmen und -mechanismen (Bei Zustimmung bitte ankreuzen und Mitarbeiter namentlich benennen)							
Verschlüsselte oder persönliche Weitergabe von Informationen?			Zustimmung zum QS-Check der Arbeit durch ... ¹ andere Mitarbeiter				
Arbeit enthält der Schutzklasse entsprechenden Sperrvermerk?			Name/n				
Freigabe der Arbeit durch den Information-Owner zur Weitergabe an die Hochschule zu Prüfungszwecken am:							
Rückgabe oder Vernichtung ausgehändigter Informationen, Betriebs- und Arbeitsmittel am:							
Veröffentlichung der Arbeit möglich?		nein	mit Schwärzung	in Teilen	uneingeschränkt		
ggf. ergänzende Vereinbarung zur Veröffentlichung:							

¹ Bei Studienarbeiten der Sicherheitsstufe „VERTRAULICH“ kann der Information-Owner im Einzelfall einen Qualitätscheck durch Mitarbeiter, die in die vertraulichen Themen involviert sind, freigeben.

Für die Arbeit genutzte Dokumente/Informationen und/oder Zugangsberechtigung für den Zugang zu sensiblen Bereichen und best. Informationen:

Falls zutreffend bitte ankreuzen:
Ich bestätige, dass ich ausschließlich Informationen der Klassifizierung "OFFEN" oder "INTERN" aus intern frei verfügbaren Medien verwendet habe.
(Die Unterschrift eines Information-Owners wird nicht zwingend benötigt.)

Unterschrift oder digitale Signatur Ersteller der Studienarbeit

Unterschrift oder digitale Signatur Information-Owner

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Glossar	iii
Tabellenverzeichnis	iv
1 Einleitung	1
1.1 Relevanz und Zielsetzung.....	1
1.2 Problemstellung und Forschungsfrage	1
1.3 Aufbau der Arbeit.....	1
2 Theoretischer Hintergrund.....	2
2.1 Datenvisualisierung in der Praxis	2
2.2 Vorstellung der Bibliotheken Matplotlib und Seaborn.....	2
2.3 Anforderungen an Visualisierungsbibliotheken	3
3 Methodisches Vorgehen	4
3.1 Auswahl und Operationalisierung der Vergleichskriterien	4
3.2 Vorgehen bei der Bewertung der Benutzerfreundlichkeit.....	5
3.3 Vorgehen bei der Analyse des Funktionsumfangs	6
4 Vergleich von Matplotlib und Seaborn	7
4.1 Analyse der Benutzerfreundlichkeit	7
4.2 Analyse des Funktionsumfangs und Anpassungsmöglichkeiten.....	12
4.2.1 Anzahl der verfügbaren Visualisierungstypen	12
4.2.2 Möglichkeiten der individuellen Anpassung.....	13
4.3 Objektiver Vergleich	15
4.4 Empfehlung für typische Anwendungsfälle	17
5 Fazit	18
5.1 Beantwortung der Forschungsfrage.....	18
5.2 Grenzen der Untersuchung	18
5.3 Ausblick.....	19
Literaturverzeichnis	21
KI-Verzeichnis	22
Anhang.....	23
Ehrenwörtliche Erklärung.....	27

Abbildungsverzeichnis

Abbildung 1: Scatterplots in Seaborn und Matplotlib	9
Abbildung 2: Bewertungsmatrix Matplotlib vs. Seaborn	16
Abbildung 3: API-Konsistenz / Seaborn vs. Matplotlib	24
Abbildung 4: Beispielhafte Plotvielfalt: Seaborn vs. Matplotlib	26
Abbildung 5: Unterschiedliche Anpassungslogik: Farbe	26
Abbildung 6: Seaborn: FacetGrid (age nach Geschlecht)	27
Abbildung 7: Matplotlib: Subplots (age nach Geschlecht)	27

Glossar

Begriff	Definition / Erläuterung
DataFrame	Ein DataFrame ist eine zentrale Datenstruktur innerhalb der Pandas-Bibliothek. Er stellt eine zweidimensionale Tabelle dar, die sich durch flexible Indizierung, heterogene Datentypen und zahlreiche Analysefunktionen auszeichnet. ¹
Jupyter Notebook	Jupyter Notebook ist eine schlanke, interaktive Entwicklungsumgebung zur Erstellung sogenannter <i>computational notebooks</i> . Diese kombinieren Quellcode, erklärende Texte, Daten, Visualisierungen und interaktive Elemente in einem Dokument. ²
Maschinelles Lernen	Ein Teilgebiet der Künstlichen Intelligenz, bei dem Computer aus Daten lernen, Muster erkennen und auf dieser Basis Vorhersagen oder Entscheidungen treffen – ohne explizit programmiert zu sein. ³

¹ Vgl. Pandas-Dokumentation o.J., o.S. Abschnitt: pandas.DataFrame.

² Vgl. Jupyter Documentation o.J., o.S.

³ Vgl. Alpaydin 2020, S. 2ff.

Plotting Plotting bezeichnet den Prozess der Erstellung von Grafiken zur Visualisierung von Daten.⁴

Pandas Pandas ist eine Open-Source-Python-Bibliothek zur Datenanalyse, die strukturierte Daten in Form von Tabellen effizient verarbeiten, filtern und analysieren kann. Sie ist insbesondere für numerische Daten geeignet und bildet die Grundlage vieler Analyseschritte in der Datenwissenschaft⁵

Tabellenverzeichnis

Tabelle 1: Operationalisierung der Kriterien.....	5
Tabelle 2: Bewertung Effektivität	8
Tabelle 3: Bewertung Erlernbarkeit	10
Tabelle 4: Bewertung Fehlertoleranz	11
Tabelle 5: Bewertung API-Konsistenz	12
Tabelle 6: typische Plottypen in den Bibliotheken.....	13
Tabelle 7: Vergleich zentraler Anpassungsaspekte in Matplotlib und Seaborn.....	14
Tabelle 8: Bewertung der Benutzerfreundlichkeit.....	15
Tabelle 9: Bewertung des Funktionsumfangs	16
Tabelle 10: Handlungsempfehlung nach Anwendungskontext.....	17
Tabelle 11: Ausschnitt der verwendeten Daten	23
Tabelle 12: Übersicht aller nativen Plottypen	25

⁴ Vgl. Pyplot tutorial o.J., o.S.

⁵ Vgl. Pandas-Dokumentation o.J., o.S.

1 Einleitung

1.1 Relevanz und Zielsetzung

In der Datenanalyse gewinnt visuelle Aufbereitung von Informationen zunehmend an Bedeutung. Im Betrieb helfen Datenvisualisierungen dabei, Kunden Sachverhalte verständlich zu verpacken. Die Programmiersprache Python hat sich für Datenanalyse und maschinellem Lernen etabliert und bietet mit den zahlreichen Bibliotheken, viele Möglichkeiten zur Erstellung von Diagrammen und diversen Grafiken. Zwei weitverbreitete Bibliotheken, zur Erstellung dieser Grafiken sind Seaborn und Matplotlib. Sie bieten verschiedene Ansätze und Funktionsspektren. Ziel dieser Arbeit ist es, einen systematischen Vergleich der beiden Bibliotheken durchzuführen, um die Eignung der Bibliotheken auf unterschiedliche Anwendungskontexte zu bewerten.

1.2 Problemstellung und Forschungsfrage

Matplotlib gilt als etablierter Standard, wobei aber Seaborn oft mit höherer Abstraktion und Einfachheit punktet. Obwohl beide Bibliotheken häufig verwendet werden, besteht Unsicherheit darüber, für welche Anwendungsfälle die Bibliotheken jeweils besser geeignet sind. Daraus ergibt sich die Zentrale Frage dieser Arbeit: **Welche der beiden Bibliotheken – Matplotlib oder Seaborn – ist unter Berücksichtigung von Benutzerfreundlichkeit und Funktionsumfang besser geeignet für die datenanalytische Praxis?**

1.3 Aufbau der Arbeit

Kapitel 2 „Theoretischer Hintergrund“ stellt die Grundlagen vor und definiert die Vergleichskriterien für den Gesichtspunkt „Benutzerfreundlichkeit“. Kapitel 3 beschreibt die Methodische Vorgehensweise. In Kapitel 4 erfolgt der direkte Vergleich der Bibliotheken. Kapitel 5 fasst die Ergebnisse zusammen und leitet konkrete Empfehlungen ab, bevor die Arbeit mit Kapitel 6, dem Fazit abgeschlossen wird.

Hinweis zur sprachlichen Gleichbehandlung: Zur besseren Lesbarkeit wird in dieser Arbeit das generische Maskulinum verwendet. Alle personenbezogenen Bezeichnungen gelten gleichermaßen für alle Geschlechter.

2 Theoretischer Hintergrund

2.1 Datenvisualisierung in der Praxis

Visuelle Aufbereitung von Daten ist ein zentraler Schlüssel moderner Datenanalyse. Sie wird genutzt, komplexe Informationen zu strukturieren, und relevante Muster, Zusammenhänge und Auffälligkeiten innerhalb umfangreicher Datensätze zu identifizieren.⁶ Grafische Darstellungen gelten als besonders effektiv, da Menschen visuelle Inhalte schneller und nachhaltiger verarbeiten als rein text- oder numerisch basierte Informationen.⁷

Im Betrieb umfasst die Anwendung der Datenvisualisierung die explorative Datenanalyse, die Präsentation wissenschaftlicher Ergebnisse sowie Unterstützung von Entscheidungen im Business Kontext. Die Visualisierung deckt dabei sowohl die analytische als auch erklärende Funktion. Einerseits fördert sie die Hypothesenbildung durch visuelle Exploration und auf der anderen Seite hilft sie bei der nachvollziehbaren Darstellung von Daten gegenüber Kunden etc.⁸

Die breite Verfügbarkeit digitaler Tools hat die Erstellung von Visualisierungen erleichtert und damit die Präsenz grafischer Darstellungen in Wissenschaft, Wirtschaft und Medien erheblich erhöht.⁹ Parallel steigt die Notwendigkeit, diese Visualisierungen zielgerichtet und überlegt zu gestalten. Ungeeignete Diagrammtypen, mangelhafte Skalierung oder überfüllte Visualisierungen können die Aussagekraft erheblich beeinträchtigen.¹⁰

Für eine wirksame Visualisierung ist es wichtig das Analyseziel, sowie die Datenaufbereitung im Voraus präzise durchzuführen. Das Messniveau der zugrunde liegenden Variablen, sowie das Analyseziel der Darstellung haben großen Einfluss auf die Auswahl geeigneter statistischer Visualisierungsformen.¹¹

2.2 Vorstellung der Bibliotheken Matplotlib und Seaborn

Die Programmiersprache Python liefert eine Vielzahl an Bibliotheken für die Erstellung von grafischen Auswertungen und Diagrammen. Zu den am weitesten verbreiteten zählen Matplotlib und Seaborn. Die beiden Bibliotheken unterscheiden sich deutlich in den Punkten ihrer Zielsetzung, Benutzeroberfläche und technischen Architektur.

⁶ Vgl. Frey 2017, S. 2.

⁷ Vgl. Frey 2017, S. 2.

⁸ Vgl. Frey 2017, S. 4.

⁹ Vgl. Frey 2017, S. 2.

¹⁰ Vgl. Frey 2017, S. 2.

¹¹ Vgl. Frey 2017, S. 4f.

Matplotlib gilt als grundlegendes Framework für die Erstellung von Grafiken und Diagrammen. Es ermöglicht präzise Steuerung einzelner Elemente und eignet sich besonders gut, wenn eine hohe Anpassbarkeit gefragt ist. Unterstützt werden unter anderem Balken-, Linien-, Kreis- und Punktdiagramme sowie Histogramme und Flächendiagramme.¹² Diese Eigenschaft macht Matplotlib besonders für technisch versierte Nutzer attraktiv, erfordert jedoch aufgrund von hoher Komplexität ein gutes Verständnis des Code-Aufbaus.¹³

Seaborn erweitert Matplotlib um vorgefertigte Funktionen für statistische Analysen. Sie erlaubt eine interaktivere und ansprechendere Darstellung komplexer Zusammenhänge. Dies vor allem durch die Verwendung integrierter Datensätze und optimierter Standarddesigns.¹⁴ Funktionen wie Displot oder das Visualisieren von Korrelationen in großen Datensätzen ermöglichen eine systematische Aufbereitung von Analyseergebnissen mit wenig Codeaufwand.¹⁵

2.3 Anforderungen an Visualisierungsbibliotheken

Die Auswahl einer geeigneten Visualisierungsbibliothek hängt wesentlich von den Anforderungen ab, die sich aus dem geplanten Einsatzzweck, der Zielgruppe und den vorliegenden Daten ergeben. Für die Bewertung der Eignung bieten sich mehrere Dimensionen an, die sowohl auf formale Kriterien als auch auf nutzerorientierte Aspekte Bezug nehmen.

Wesentliche Anforderungen betreffen zunächst die Funktionalität und Ausdrucksstärke einer Bibliothek. Dazu gehören Aspekte wie die Unterstützung verschiedener Diagrammtypen, die Möglichkeit der Darstellung mehrdimensionaler Daten, sowie die Flexibilität bei Layout und Beschriftung. Eine Visualisierungsbibliothek sollte zu einem explorative und zum anderen erklärende Darstellungen ermöglichen und eine verständliche Übertragung von Dateneigenschaften in visuelle Merkmale gewährleisten.¹⁶

Ein weiterer zentraler Aspekt ist Benutzerfreundlichkeit. Aufbauend auf dem Modell von Shackle lassen sich folgende Merkmale zur Bewertung heranziehen: Effektivität, Erlernbarkeit, Flexibilität und Akzeptanz (subjektive Bewertung durch den Nutzer).¹⁷ Diese Merkmale können in adaptierter Form auf Softwarebibliotheken übertragen werden.

¹² Vgl. Sial et al. 2021, S. 280.

¹³ Vgl. Sial et al. 2021, S. 280.

¹⁴ Vgl. Sial et al. 2021, S. 280.

¹⁵ Vgl. Sial et al. 2021, S. 281.

¹⁶ Vgl. Frey 2018, S. 2ff.

¹⁷ Vgl. Richter 1997, S. 10ff.

Darüber hinaus verweist Richter auf die Relevanz normativer Gestaltungsprinzipien, wie sie in internationalen Standards zur Gestaltung interaktiver Systeme formuliert sind. Diese umfassen beispielweise Aufgabenangemessenheit, Erwartungskonformität, Fehlertoleranz, Kompatibilität, Individualisierbarkeit und Steuerbarkeit.¹⁸ Es entsteht ein Bewertungsschema aus nutzerorientierten und wissenschaftlich gestützten Anforderungen, welches sich zur Bewertung von Visualisierungsbibliotheken für die Datenanalyse gut eignet.

Ein weiterer wichtiger Aspekt ist die Kompatibilität mit Datenformaten und Analyseumgebungen. Bibliotheken sollten mit gängigen Datenstrukturen (z.B. Pandas-DataFrames) arbeiten können und sich nahtlos in bestehende Analyseprozesse einfügen. Für wissenschaftliche Anwendungsfelder ist zudem die Reproduzierbarkeit der erzeugten Darstellungen von Bedeutung.

3 Methodisches Vorgehen

3.1 Auswahl und Operationalisierung der Vergleichskriterien

Für den objektiven Vergleich der beiden Bibliotheken wurde eine Auswahl wissenschaftlich fundierter Bewertungskriterien geführt. Die Kriterien sind von anerkannten Modellen der Softwareergonomie sowie Anforderungen der Datenanalyse in der Praxis entnommen. Grundlage sind die Prinzipien der ISO 9241-110, wie sie in frei zugänglicher Literatur erläutert werden¹⁹, als auch das Modell von Shackel (1991), welches von Richter (1997) konkretisiert wird.²⁰

Die Kriterien wurden so gewählt, dass sie sich rein objektiv erheben lassen, um ein wissenschaftlich fundiertes Ergebnis zu gewährleisten. Die Bewertung gewährleistet somit die Reproduzierbarkeit und Nachvollziehbarkeit. Ziel ist es, sowohl die Benutzerfreundlichkeit als auch den Funktionsumfang beider Bibliotheken systematisch zu bewerten. Die nachfolgende Tabelle 1 fasst die Kriterien, ihre theoretische Fundierung und ihre Operationalisierung zusammen.

¹⁸ Vgl. Richter 1997, S. 14, Tabelle 3.

¹⁹ Vgl. Handbuch Usability o.J., o.S.

²⁰ Vgl. Richter 1997, S. 11-14.

Tabelle 1: Operationalisierung der Kriterien

Kriterium	Bezug auf:	Objektive Operationalisierung
Effektivität	Shackel-Modell	Anzahl der Codezeilen zur Erstellung eines Standardplots
Erlernbarkeit	ISO 9241-110, Shackel	Schritte bis zur ersten erfolgreichen Visualisierung / Gesamtkomplexität
Fehlertoleranz	ISO 9241-110	Reaktion auf typische Fehler (z.B. fehlende Argumente)
API-Konsistenz	Erwartungskonformität nach ISO und Richter	Einheitlichkeit von Funktions- und Argumentnamen in der Doku
Funktionsumfang	datenanalytische Praxis	Anzahl der in der Dokumentation gelisteten Visualisierungstypen

3.2 Vorgehen bei der Bewertung der Benutzerfreundlichkeit

Die Bewertung der Benutzerfreundlichkeit erfolgt anhand der definierten Kriterien (siehe. 3.1). Ziel ist es, beide Bibliotheken unter kontrollierten Bedingungen so zu bewerten, dass der Vergleich sowohl objektiv als auch nachvollziehbar ist.

Effektivität: Die Effektivität wird bestimmt durch die Anzahl der notwendigen Codezeilen, um eine Standardvisualisierung zu erzeugen – Zum Beispiel ein Histogramm auf Basis eines DataFrames. Dieses Vorgehen orientiert sich an Shackel's Modell, in dem Effektivität als das Maß gilt, mit dem ein Nutzer die Zielaufgabe korrekt und vollständig erledigen kann. Je weniger Schritte erforderlich sind, desto höher ist in diesem Zusammenhang die Effektivität.²¹

Erlernbarkeit: Die Erlernbarkeit wird über die Anzahl der notwendigen Schritte bis zur Erstellung einer Grafik bestimmt. Darunter fallen die Schritte wie Importieren der Bibliothek, die Vorbereitung der Daten und der Einsatz eines geeigneten Funktionsbefehls. Die Bewertung folgt der Definition bei Richter, wonach Erlernbarkeit anhand der Anzahl von Versuchen oder benötigten Hilfestellungen gemessen werden kann.²²

²¹ Vgl. Richter 1997, S. 10.

²²Vgl. Richter 1997, S. 10.

Fehlertoleranz: Um die Fehlertoleranz zu bewerten, wird gezielt falscher oder unvollständiger Code eingegeben – etwas das Weglassen von Pflichtparametern oder das Verwenden ungültiger Argumente. Danach wird dann bewertet, ob die Bibliothek eine klare, verständliche Fehlermeldung erzeugt und ob diese dem Nutzer zur Fehlerbehebung verhilft. Ein System ist dann fehlertolerant, wenn es bei fehlerhaften Eingaben weiterhin korrekt oder unterstützend reagiert.²³

API – Konsistenz: Die Konsistenz der API wird durch die Analyse der offiziellen Dokumentation beider Bibliotheken untersucht. Überprüft wird, ob gleichartige Funktionen konsistente Argumente und Benennungen verwenden, was dem Anwendungskonform entgegenkommt. Erwartungskonformität beschreibt den Sachverhalt, dass ein System Erwartungen an Verlässlichkeit und Vorhersagbarkeit erfüllt.²⁴

Diese Methodik erlaubt eine systematische Beurteilung der Bibliotheken. Die konkreten Ergebnisse der Bewertung werden in Kapitel 4.1 dargestellt und durch praktische Beispiele ergänzt.

3.3 Vorgehen bei der Analyse des Funktionsumfangs

Die Bewertung des Funktionsumfangs von Matplotlib und Seaborn wird zweistufig geschehen. Ziel ist es, die Breite und Tiefe der Bibliotheken objektiv zu erfassen und vergleichbar darzustellen. Dabei werden zwei Dimensionen berücksichtigt: erstens die Vielfalt an unterstützten Diagrammtypen und zweitens der Grad an individueller Anpassbarkeit.

Zuerst erfolgt eine Dokumentenanalyse der offiziellen Bibliotheksdokumentationen. Für beide Bibliotheken wird dabei evaluiert, welche spezifischen Visualisierungstypen nativ unterstützt werden. Folgend wird untersucht, in welchem Maße die jeweiligen Diagramme individuell anpassbar sind. Hierzu wird die Modifizierbarkeit der Diagramme bewertet, indem geprüft wird inwiefern die Diagramme über Parameter und Zusatzfunktionen nach Wunsch anpassbar sind, und ob, oder zusätzliche Programmierlogik erforderlich ist. Zur Veranschaulichung werden BeispielpLOTS, und deren Erstellung in Code dokumentiert und kurz erläutert.

²³ Vgl. Handbuch Usability o.J., o.S.

²⁴ Vgl. Richter 1997, S. 12.

4 Vergleich von Matplotlib und Seaborn

4.1 Analyse der Benutzerfreundlichkeit

Alle im Folgenden dargestellten Codebeispiele und Visualisierungen basieren auf dem Titanic-Datensatz, der öffentlich zugänglich ist. Für die Vergleichbarkeit wird für beide Bibliotheken derselbe Datensatz verwendet. Für die Auswertung wurde eine CSV-Version dieses Datensatzes lokal gespeichert und die Codierung der Diagramme erfolgt in einem Jupyter Notebook. Alle Codebeispiele werden in dieser Umgebung ausgeführt. Die in der genannten Umgebung erstellten Plots, wurden unverändert in diese Arbeit übernommen.

Codebeispiel 1: Einlesen der CSV in Python Dataframe

```
import pandas as pd #importieren der Bibliothek
titanic = pd.read_csv("titanic.csv") #CSV in Pandas einlesen
```

Ein repräsentativer Auszug der verwendeten Daten ist in Anhang A dargestellt.²⁵

Zur Analyse der Benutzerfreundlichkeit werden die Bibliotheken anhand der in Kapitel 3.1 definierten Kriterien nach einem einheitlichen Schema bewertet. Grundlage sind typische Aufgaben, die jeweils mit beiden Bibliotheken gelöst werden. Die Bewertung erfolgt anhand der konkreten Umsetzung.

Effektivität:

Aufgabe: Ein Scatterplot mit den Spalten *age* (x-Achse) und *fare* (y-Achse), wobei die Farbe der Punkte das Geschlecht (*sex*) widerspiegelt. Der Plot soll eine automatische Legende und beschriftete Achsen enthalten. Bewertet wird die Zielerreichung mit minimalem Aufwand.

Codebeispiel 2: Effektivität Seaborn

```
# Das "#" initiiert in Python einen Kommentar
# scatterplot: x = Alter, y = Ticketpreis
# hue = Farbliche Trennung nach Geschlecht (automatische Legende)
sns.scatterplot(data=titanic, x="age", y="fare", hue="sex")
```

²⁵ Vgl. Titanic Dataset o.J., o.S.

Codebeispiel 3: Effektivität Matplotlib

```
# Erstellt ein leeres Plot-Fenster mit Achsenobjekt
fig, ax = plt.subplots()

# Manuelle Farbzuzuordnung je Geschlecht
colors = {"male": "blue", "female": "red"}

# Scatterplot mit Farbcodierung aus Mapping
ax.scatter(titanic["age"], titanic["fare"], c=titanic["sex"].map(colors))

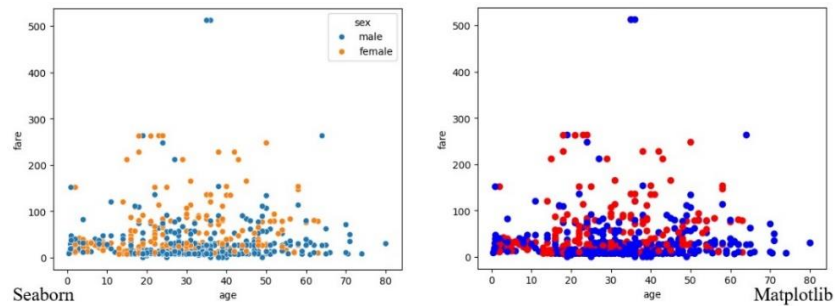
# Achsentitel hinzufügen
ax.set_xlabel("age")
ax.set_ylabel("fare")
```

Tabelle 2: Bewertung Effektivität

Bibliothek	Punkte (1-5)	Begründung
Seaborn	5	Der Scatterplot wird mit einem einzigen Funktionsaufruf erzeugt (Codebeispiel 2). Dabei werden Achsentitel als auch Farbzuzuweisung und Legende automatisch erstellt – allein durch die Parameter <code>x</code> , <code>y</code> und <code>hue</code> . Ein zusätzlicher Konfigurationsaufwand entfällt vollständig.
Matplotlib	2	Das Plot-Fenster muss manuell erstellt werden, ebenso wie die Farbzuzuweisung und die Achsentitel. Eine Legende fehlt vollständig ²⁶ , da Matplotlib diese in Kombination mit Farbmapping nicht automatisch erzeugt. Für eine äquivalente Darstellung müsste zusätzlicher Code ergänzt werden. (Codebeispiel 3)

²⁶ Siehe Abbildung 1: Scatterplots in Seaborn und Matplotlib.

Abbildung 1: Scatterplots in Seaborn und Matplotlib

**Erlernbarkeit:**

Aufgabe: Ein gruppiertes Balkendiagramm zur Anzahl Überlebender (*survived*) nach Geschlecht (*sex*) aufgeschlüsselt.

Codebeispiel 4: Erlernbarkeit Seaborn

```
# Einfache Gruppierung durch Parameterkombination
sns.countplot(data=titanic, x="sex", hue="survived")
```

Codebeispiel 5: Erlernbarkeit Matplotlib

```
import numpy as np # Standard Mathe Bibliothek in Python

# Gruppierte Zählung über crosstab
grouped = pd.crosstab(titanic["sex"], titanic["survived"])

# X-Achsen-Positionen
x = np.arange(len(grouped.index)) # z.B. [0, 1]
width = 0.35

# Balken zeichnen
plt.bar(x - width/2, grouped[0], width=width, label="Not Survived")
plt.bar(x + width/2, grouped[1], width=width, label="Survived")

# Achsentitel, xticks und Legende
plt.xticks(x, grouped.index)
plt.xlabel("sex")
plt.ylabel("count")
plt.legend()
```

Tabelle 3: Bewertung Erlernbarkeit

Bibliothek	Punkte (1-5)	Begründung
Seaborn	5	Es genügt ein Funktionsaufruf, um die Gruppierung und Darstellung zu erledigen. Die Parameter sind semantisch und direkt verständlich (x, hue). (Codebeispiel 4)
Matplotlib	2	Es sind mehrere Zwischenschritte nötig, um Daten zu gruppieren und darzustellen. Dazu gehört eine Zählung mit <code>pd.crosstab</code> , Positionierung der Balken mit NumPy, manuelles Setzen von <code>xticks</code> , Labels und Legende. Für Einsteiger ist diese Konstruktion deutlich aufwendiger und erfordert Verständnis für mehrere Bibliotheken. Die Vorverarbeitung erschwert die Verständlichkeit und intuitive Programmierung erheblich. (Codebeispiel 5)

Fehlertoleranz:

Aufgabe: typische Fehlersimulation: Zugriff auf eine nicht existierende Spalte (agge statt age). Bewertet wird die Qualität der Fehlermeldung.

Codebeispiel 6: Fehlertoleranz Seaborn

```
# Fehlerhafte Eingabe - Spalte 'agge' existiert nicht
sns.histplot(data=titanic, x="agge")
# Ausgabe: ValueError: "Could not interpret value 'agge' for 'x'.
An entry with this name does not appear in 'data'.
```

Codebeispiel 7: Fehlertoleranz Matplotlib

```
# Fehlerhafte Eingabe - KeyError
plt.hist(titanic["agge"])
# Ausgabe: KeyError - 'agge'
```

Tabelle 4: Bewertung Fehlertoleranz

Bibliothek	Punkte (1-5)	Begründung
Seaborn	4	Die Fehlermeldung benennt explizit den Parameter <code>x</code> und den falsch Einggegebenen Wert <code>agge</code> , was die Ursache schnell verständlich macht. (Codebeispiel 6)
Matplotlib	3	Die Fehlermeldung in ist technisch korrekt, aber wenig hilfreich für Einsteiger, da der Parameterbezug fehlt. (Codebeispiel 7)

API-Konsistenz:

Aufgabe: Drei verschiedene Diagrammtypen die *inhaltlich gleich* sind.

Codebeispiel 8: API-Konsistenz Seaborn

```
# Histogramm: Verteilung Alter
sns.histplot(data=titanic, x="age")
# Boxplot: Preisverteilung nach Klasse
sns.boxplot(data=titanic, x="class", y="fare")
# Scatterplot: Alter vs. Preis, farblich nach Geschlecht
sns.scatterplot(data=titanic, x="age", y="fare", hue="sex")
```

Codebeispiel 9: API-Konsistenz Matplotlib

```
# Histogramm
plt.hist(titanic["age"])
# Boxplot - keine Parameter wie x/y, sondern Listenstruktur
plt.boxplot([titanic[titanic["class"] == "First"]["fare"],
             titanic[titanic["class"] == "Second"]["fare"],
             titanic[titanic["class"] == "Third"]["fare"]])
plt.xticks([1, 2, 3], ["First", "Second", "Third"])
# Scatterplot
plt.scatter(titanic["age"], titanic["fare"])
```


Tabelle 5: Bewertung API-Konsistenz

Bibliothek	Punkte (1-5)	Begründung
Seaborn	5	Die API ist in allen drei Beispielen konsistent. Die Struktur „ <i>data=..., x=..., y=..., hue=...</i> “ bleibt gleich. Dadurch bleiben Konzepte durch den Code hinweg wiederaufgreifbar. (Codebeispiel 8)
Matplotlib	2	Die Funktionsaufrufe „ <i>plt.hist, plt.boxplot, plt.scatter</i> “ unterscheiden sich stark. Parameterstrukturen, Eingabelogik und Kontextverhalten (z.B. <i>xticks</i> setzen) müssen für jede Plotart neu verstanden werden. Eine durchgängige API-Logik fehlt. (Codebeispiel 9)

Außerdem unterscheiden sich die Diagramme sehr in ihrer Darstellungsweise, wobei Seaborn hier auch, durch besser differenzierbare Balken und Datenpunkte, in der Betrachtung punktet.

4.2 Analyse des Funktionsumfangs und Anpassungsmöglichkeiten

4.2.1 Anzahl der verfügbaren Visualisierungstypen

Ein zentrales Kriterium beim Vergleich von den Bibliotheken stellt die Vielfalt der unterstützten Diagrammtypen dar. Seaborn und Matplotlib unterscheiden sich sowohl in der Zielsetzung als auch in technischer Architektur. Wo Matplotlib als Low-Level-Bibliothek eine besonders breite Palette an Standarddiagrammen durch manuelle Konfiguration erlaubt, setzt Seaborn auf eine Auswahl statistisch orientierter Darstellungen mit höherer Abstraktion.

Matplotlib unterstützt eine Vielzahl von Basisdiagrammen wie Linien-, Balken-, Kreisdiagramme oder Histogramme. Dank ihres modularen Aufbaus lassen sich darüber hinaus auch komplexere Visualisierungen erzeugen. Diese Vielfalt geht jedoch mit einem gewissen Programmieraufwand einher, da viele Einstellungen manuell vorgenommen werden müssen.²⁷ Seaborn bietet dagegen eine gezielte Auswahl typischer Diagrammtypen, die insbesondere für explorative Datenanalysen optimiert sind. Hierzu zählen unter anderem Scatterplots mit Regressionslinien (*regplot*), Dichteplots (*kdeplot*), Kategoriediagramme (*boxplot*, *violinplot*) sowie matrixbasierte Visualisierungen (*heatmap*, *pairplot*). Die Visualisierungen lassen sich dabei

²⁷ Vgl. Han/Kwak 2023, S.168.

direkt aus den Pandas-DataFrames erzeugen und sind mit semantischen Parametern wie `hue`, `style` oder `col` einfach steuerbar.²⁸

Im direkten Vergleich zeigt sich, dass Seaborn durch die automatische Umsetzung statistischer Logiken und komfortabler Standarddesigns überzeugt. Die Deklaration der Diagrammtypen erfolgt mit deutlich weniger Codezeilen, was die Anwendung insbesondere in der explorativen Phase beschleunigt.²⁹

Unter dem Aspekt, dass für die Datenanalyse nicht alle Plottypen gebraucht werden, zeigt die nachfolgende Tabelle eine exemplarische Auswahl typischer Plottypen, die anhand der offiziellen Dokumentationen beider Bibliotheken abgeleitet wurde. Eine vollständige Übersicht aller dokumentierten nativen Plotfunktionen und eine Auswahl beispielhaft umgesetzter Visualisierungen, sind ergänzend im Anhang aufgeführt. (Tabelle 12/ Abbildung 4/ Abbildung 5/ Abbildung 6/ Abbildung 7). Nativ bedeutet in dem Zusammenhang, dass es für den jeweiligen Plot eine eigene Funktion gibt, die ohne externe Bibliotheken oder Subklassen umsetzbar ist.

Tabelle 6: typische Plottypen in den Bibliotheken

<i>Seaborn (Auswahl)</i>	<i>Matplotlib (Auswahl)</i>
<i>Histogramm (histplot)</i>	Histogramm (hist)
<i>Scatterplot (scatterplot)</i>	Scatterplot (scatter)
<i>Boxplot (boxplot)</i>	Boxplot (boxplot)
<i>Heatmap (heatmap)</i>	Heatmap (mit <code>imshow()</code> , manuelle skaliert)
<i>Pairplot (pairplot)</i>	Kein direktes Äquivalent (manuell)
<i>Violinplot (violinplot)</i>	Violinplot (violinplot)

4.2.2 Möglichkeiten der individuellen Anpassung

Neben dem Funktionsumfang hat die Anpassbarkeit von Visualisierungen auch eine entscheidende Rolle für den praktischen Einsatz. Dabei umfasst „Anpassung“ insbesondere das gezielte Verändern von Achsentiteln, Farben, Stilmitteln, Layouts, Mehrfachplots etc.

²⁸ Vgl. Waskom 2021, S. 1f.

²⁹ Vgl. Rashdi et al. 2021, S. 278

Während Matplotlib nahezu unbegrenzte Anpassungsmöglichkeiten bietet, erfordert dies aber auch einen höheren Konfigurationsaufwand. Jede Änderung, zum Beispiel eine Achsenskalierung, die Schriftgröße oder die Farbpalette, muss explizit im Code definiert werden. Dagegen verfolgt Seaborn einen deklarativen Ansatz, bei dem viele dieser Merkmale schon automatisch gesetzt oder über einfache Parameter wie *hue*, *style*, *col* oder *palette* gesteuert werden können. Dies kommt einem unerfahrenem Nutzer zugunsten, schränkt aber erfahrene Nutzer in ihrer Freiheit ein. Folgende Tabelle hält die Anpassungsaspekte in Seaborn³⁰ und Matplotlib³¹ übersichtlich fest.

Tabelle 7: Vergleich zentraler Anpassungsaspekte in Matplotlib und Seaborn

Anpassungsaspekt	Matplotlib	Seaborn
Achsentitel / Labels	Vollständig manuell über <i>sex_xlabel()</i> etc.	Automatisch + überschreibbar (x, y)
Farben / Paletten	Frei definierbar über <i>color</i> , <i>cmap</i> , <i>set_prop_cycle()</i>	Semantisch zuweisbar (<i>hue</i> , <i>palette</i>)
Subplots / Grids	Manuelle Achsenverwaltung mit <i>subplots()</i>	Automatisch über <i>FacetGrid</i> , <i>catplot()</i>
Stil / Theme	Anpassung via <i>rcParams</i> , <i>style.use()</i>	Globale Themes (<i>set_theme()</i>)
Beschriftungen / Legende	Manuell (<i>legend()</i> , <i>annotate()</i>)	Automatisch, ggf. anpassbar
Layout / Größe	Frei skalierbar mit <i>figsize</i> , <i>tight_layout()</i>	Parameter wie <i>height</i> , <i>aspect</i>

³⁰ Vgl. API-reference, o.J., o.S.

³¹ Vgl. Plot types, o.J., o.S.

4.3 Objektiver Vergleich

Dieses Kapitel fasst die Ergebnisse der vorangegangenen Analyseabschnitte zusammen. Die Bewertung erfolgt entlang der in Kapitel 3 definierten Kriterienbereiche: Benutzerfreundlichkeit (Kapitel 4.1) und Funktionsumfang inklusive Individualisierbarkeit (Kapitel 4.2). Auf Basis der objektiv ermittelten Merkmale wird eine systematische Gegenüberstellung durchgeführt.

Die Nachfolgenden Tabellen führen die Bewertungen strukturiert auf. Grundlage sind die dokumentierten Vergleiche beider Bibliotheken.

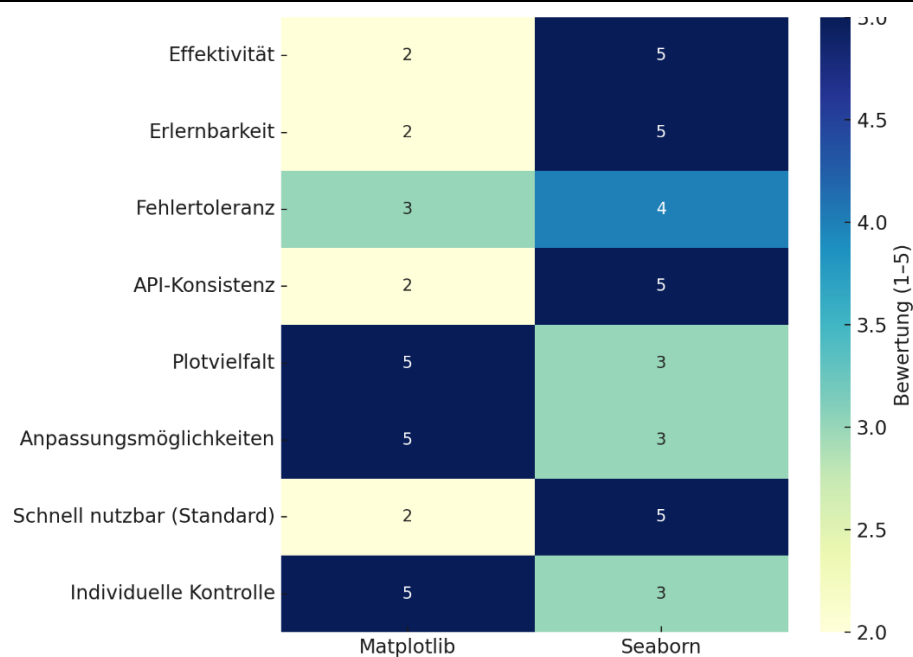
Tabelle 8: Bewertung der Benutzerfreundlichkeit

Kriterium	Matplotlib	Seaborn
Effektivität	X (mehr Code notwendig, keine automatische Legende)	✓ (Standardplots mit wenigen Parametern)
Erlernbarkeit	X (mehrstufiger Aufbau, API teils uneinheitlich)	✓ (einheitliche Struktur, semantische Parameter)
Fehlertoleranz	(✓) (technisch korrekt, aber wenig hilfreich)	✓ (erklärende Fehlermeldungen, Parameterbezug)
API-Konsistenz	X (abweichende Syntax bei Plotfunktionen)	✓ (einheitliche Funktionsstruktur)

Tabelle 9: Bewertung des Funktionsumfangs

Kriterium	Matplotlib	Seaborn
Anzahl unterstützter Diagrammtypen	✓ (sehr umfangreich, inkl. technischer Plots)	(✓) (fokussiert auf statistische Visualisierungen)
Nativ unterstützte Mehrfachplots	(✓) (über Subplot-Logik möglich)	✓ (über FacetGrid, catplot etc.)
Semantische Steuerung (z. B. Farbe)	✗ (manuell über Mapping, kein hue)	✓ (hue, style, size etc.)
Layout- und Stilanpassung	✓ (vollständig über rcParams etc.)	(✓) (vorgegebene Themes mit Parametern)
Automatische Achsenbeschriftung	✗ (immer manuell)	✓ (automatisch aus DataFrame)
Flexibilität bei komplexen Layouts	✓ (vollständig kontrollierbar)	✗ (nur innerhalb des Grid-Systems)

Abbildung 2: Bewertungsmatrix Matplotlib vs. Seaborn



4.4 Empfehlung für typische Anwendungsfälle

Auf Grundlage, der in Kapitel 4.1 bis 4.3 analysierten Kriterien lassen sich konkrete Empfehlungen für die Nutzung von Seaborn oder Matplotlib in verschiedenen Kontexten ableiten. Die folgende Tabelle ordnet typische Anforderungen der datenanalytischen Praxis den jeweiligen Stärken der Bibliotheken zu. Dabei fließen sowohl Aspekte der Benutzerfreundlichkeit als auch des Funktionsumfangs in die Empfehlung ein.

Tabelle 10: Handlungsempfehlung nach Anwendungskontext

Anwendungsfall	Empfehlung	Begründung
Schnelle Erstellung standardisierter Plots	Seaborn	Minimaler Codeaufwand, automatische Gestaltung und einfache Parameter
Explorative Datenanalyse mit Gruppierung	Seaborn	hue, col, row und FacetGrid unterstützen visuelle Datenexploration
Detaillierte Layoutanpassung / Corporate Design	Matplotlib	Umfassende Kontrolle über Stil, Schrift, Farben und Positionierungen
Technische oder spezielle Plotty-pen (z. B. 3D, Kontur, Polar)	Matplotlib	Nur dort nativ verfügbar bzw. erweiterbar
Statistische Visualisierungen mit wenigen Zeilen Code	Seaborn	Integrierte statistische Funktionen wie regplot, kdeplot, pairplot
Hohe API-Konsistenz für Einsteiger wichtig	Seaborn	Einheitliches Argumentschema in allen Funktionen
Projekte mit Fokus auf Individualisierung	Matplotlib	Volle manuelle Steuerung über jedes Diagrammelement

Diese Empfehlungen stellen eine Orientierung dar und ersetzen keine individuelle Projektanalyse. In der Praxis kann es sinnvoll sein, beide Bibliotheken kombiniert zu nutzen – etwa indem Matplotlib zur Feinanpassung von in Seaborn erzeugten Diagrammen verwendet wird.

5 Fazit

5.1 Beantwortung der Forschungsfrage

Die in Kapitel 1.2 formulierte Forschungsfrage „*Wie unterscheiden sich Seaborn und Matplotlib hinsichtlich Benutzerfreundlichkeit und Funktionsumfang, und welche der beiden Bibliotheken ist für datenanalytische Aufgaben unter didaktischen und funktionalen Gesichtspunkten besser geeignet?*“ lässt sich aufgrund der durchgeführten Analyse differenziert beantworten. Die Ergebnisse zeigen, dass Seaborn in nahezu allen untersuchten Aspekten der Benutzerfreundlichkeit – insbesondere in Bezug auf Erlernbarkeit, API-Konsistenz und Fehlertoleranz deutlich besser abschneidet als Matplotlib. Dies ist vor allem auf die deklarative API-Struktur, die automatische Umsetzung semantischer Parameter und die durchaus übersichtliche Dokumentation zurückzuführen.³²

Bei Betrachtung des Funktionsumfangs punktet Matplotlib mit der deutlich größeren Vielfalt an Plottypen, darunter auch technische und spezifischere Darstellungen wie Polardiagramme, Konturplots oder 3D-Visualisierungen. Auch die Möglichkeiten für Anpassungen im Stil, Layout und die Steuerung einzelner Elemente sind in Matplotlib umfangreicher und individueller zu steuern.³³

Die Auswertung zeigt somit keine pauschale Überlegenheit einer Bibliothek, sondern wohl eher eine klare Unterscheidung in Bezug auf den Einsatzzweck: Für explorative Analysen, didaktische Anwendungen und schnelle Visualisierungen, vor allem in der Datenanalyse ist Seaborn aufgrund seiner Benutzerfreundlichkeit vorzuziehen. In spezifischeren Anwendungsfällen, die eine hohe Individualisierbarkeit oder spezielle Diagramme benötigen, ist Matplotlib besser geeignet. Die Bibliotheken lassen sich aufgrund dessen, dass Seaborn auf Matplotlib aufgebaut ist, problemlos kombinieren. Die Kombination beider ermöglicht in der Praxis eine flexible Nutzung der jeweiligen Stärken und bietet somit eine starke Basis für viele Anwendungsfälle.

5.2 Grenzen der Untersuchung

Trotz der systematischen Vorgehensweise und des klar definierten Vergleichsrahmens weist die Untersuchung doch begrenzende Faktoren auf. Diese betreffen sowohl die Auswahl der Kriterien als auch die methodische Umsetzung.

³² Vgl. Waskom 2021, S. 1f.

³³ Vgl. Han/Kwak 2023, S. 168ff.

Zum einen wurde der Funktionsvergleich ausschließlich auf den Bereich der zweidimensionalen Diagramme eingegrenzt. Erweiterte Anwendungen wie interaktive Plots, 3D-Diagramme oder Animationen blieben unberücksichtigt, obwohl es von Matplotlib in diesen Bereichen auch Module gibt. Gleiches gilt auch für Bibliotheken wie Plotly oder Bokeh, die außerhalb des Untersuchungsrahmens liegen. Performance der Bibliotheken bei großen Datenmengen sowie Export- und Integrationsfunktionen wurden nicht untersucht.

Zum anderen basiert die Bewertung der Benutzerfreundlichkeit auf objektiven Kriterien, die sich aus der Fachliteratur ableiten lassen. Subjektive Faktoren wie die individuelle Nutzererfahrung oder empirische Usability-Tests mit Testpersonen wurden nicht einbezogen, um die Vergleichbarkeit zu wahren. Damit bleibt offen, wie die Bibliotheken in der praktischen Anwendung tatsächlich wahrgenommen werden.

Durch die Verwendung eines exemplarischen Datensatzes, lassen sich keine allgemeingültigen Aussagen für andere Datendomänen wie Zeitreihen-, Geo- oder Netzwerkdaten ableiten.

Die Ergebnisse dieser Arbeit gelten somit im Rahmen der verwendeten Bewertungskriterien und methodischen Festlegungen. Sie können eine fundierte Orientierung bieten, ersetzen jedoch keine individuelle Analyse spezifischer Anforderungen in komplexeren Anwendungsbereichen.

5.3 Ausblick

Trotz der Eingrenzungen eröffnet die Untersuchung mehrere Perspektiven für weiterführende Auseinandersetzung mit Visualisierungsbibliotheken in Python. Dabei ergeben sich sowohl technische als auch methodische Ansätze.

Ein naheliegender Schritt wäre die Einbeziehung weiterer Bibliotheken wie Plotly, Bokeh oder Altair, um auch interaktive und webbasierte Visualisierungsansätze vergleichend zu bewerten. Dies könnte vor allem in Anwendungsfeldern relevant sein, in denen Benutzerinteraktion und Ästhetik eine zentrale Rolle spielen.

Darüber hinaus ließe sich die Bewertung der Benutzerfreundlichkeit um empirische Perspektiven erweitern. Beispielsweise über nutzerbasierte Studien, in denen Entwicklergruppen Aufgaben mit verschiedenen Bibliotheken bearbeiten und ihre Erfahrungen rückmelden. Eine solche qualitative Ergänzung würde das Thema der Arbeit sinnvoll ergänzen.

Unabhängig davon zeigt sich, dass Visualisierungsbibliotheken in Python kein *Entweder-oder* fordern, sondern viel eher ein *Sowohl-Als-Auch* ermöglichen. Gerade die Kombination der Stärken von Seaborn und Matplotlib bietet ein großes Potenzial für praxisnahe, flexible und zugleich hochwertige Datenvisualisierung – ein Ansatz, der sich auch in zukünftigen Projekten weiterverfolgen lässt.

Literaturverzeichnis

- Alpaydin, E. (2020). *Introduction to Machine Learning*. MIT Press.
- Frey, T. (2017). *Grundlagen der Datenvisualisierung*. Universität Freiburg.
- Han, S., & Kwak, I.-Y. (2023). Mastering data visualization with Python: practical tips for researchers. *JMIS*, 167 - 175. Retrieved April 3, 2025, from National Library of Medicine: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10728683/>
- Handbuch Usability. (o.J). *ISO 9241*. Retrieved April 07, 2025, from Handbuch Usability: <https://www.handbuch-usability.de/grundlagen/normen-und-standards/iso-9241/>
- Matplotlib Development Team. (2002). *Plot types*. Retrieved April 06, 2025, from matplotlib: https://matplotlib.org/stable/plot_types/index.html
- Matplotlib Development Team. (o.J). *Pyplot tutorial*. Retrieved April 06, 2025, from Matplotlib Documentation: <https://matplotlib.org/stable/tutorials/pyplot.html>
- pydata.org*. (o.J). Retrieved April 06, 2025, from Pandas Documentation: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html#pandas.DataFrame>
- Rashdi, S. Y., Khan, A. H., & Sial, A. H. (2021). *Comparative Analysis of Data Visualization Libraries Matplotlib and Seaborn in Python*. International Journal of Advanced Trends in Computer Science and Engineering.
- Richter, M. (1997). *Kriterien der Benutzerfreundlichkeit*. Käshaldenstr. 35 8052 Zürich: Philosophische Fakultät der Universität Zürich.
- Waskom, M. (2012). *API reference*, v0.13.2. Retrieved April 06, 2025, from seaborn: <https://seaborn.pydata.org/api.html#>
- Waskom, M. L. (2021). *seaborn: statistical data visualization*. Journal of Open Source Software.

KI-Verzeichnis

System	Prompt	Verwendung
GPT 4o	Schreibe mir ein passendes Inhaltsverzeichnis zu meinem Exposé.	Verändert: Kapitel entfernt / hinzugefügt
GPT 4o	Prüfe meine WAB auf Einhaltung der Proবাদis Formatvorgaben	unverändert
GPT 4o	Passe den Text an die Verwendung des generischen Maskulinums an.	Verändert / Passagen überarbeitet.
GPT 4o	Gib mir für meine gewählten Kriterien, Aufgaben, die sich mit Seaborn und Matplotlib umsetzen, und die Ausführung objektiv bewerten lassen. (WAB im Anhang)	Wortlaut angepasst / ansonsten unverändert
GPT 4o	Prüfe die Arbeit auf Grammatik und Rechtschreibfehler	Passagen überarbeitet

Anhang

Tabelle 11: Ausschnitt der verwendeten Daten

survived	sex	age	fare	embarked	class	who
0	male	22	7,25	S	Third	man
1	female	38	71,2833	C	First	woman
1	female	26	7,925	S	Third	woman
1	female	35	53,1	S	First	woman
0	male	35	8,05	S	Third	man
0	male		8,4583	Q	Third	man
0	male	54	51,8625	S	First	man
0	male	2	21,075	S	Third	child
1	female	27	11,1333	S	Third	woman
1	female	14	30,0708	C	Second	child
1	female	4	16,7	S	Third	child
1	female	58	26,55	S	First	woman
0	male	20	8,05	S	Third	man
0	male	39	31,275	S	Third	man
0	female	14	7,8542	S	Third	child
1	female	55	16	S	Second	woman
0	male	2	29,125	Q	Third	child
1	male		13	S	Second	man
0	female	31	18	S	Third	woman
1	female		7,225	C	Third	woman
0	male	35	26	S	Second	man
1	male	34	13	S	Second	man
1	female	15	8,0292	Q	Third	child
1	male	28	35,5	S	First	man
0	female	8	21,075	S	Third	child
1	female	38	31,3875	S	Third	woman
0	male		7,225	C	Third	man
0	male	19	263	S	First	man
1	female		7,8792	Q	Third	woman
0	male		7,8958	S	Third	man
0	male	40	27,7208	C	First	man
1	female		146,5208	C	First	woman
1	female		7,75	Q	Third	woman
0	male	66	10,5	S	Second	man
0	male	28	82,1708	C	First	man
0	male	42	52	S	First	man
1	male		7,2292	C	Third	man
0	male	21	8,05	S	Third	man
0	female	18	18	S	Third	woman
1	female	14	11,2417	C	Third	child

0	female	40	9,475	S	Third	woman
0	female	27	21	S	Second	woman
0	male		7,8958	C	Third	man
1	female	3	41,5792	C	Second	child
1	female	19	7,8792	Q	Third	woman
0	male		8,05	S	Third	man
0	male		15,5	Q	Third	man
1	female		7,75	Q	Third	woman
0	male		21,6792	C	Third	man

Abbildung 3: API-Konsistenz / Seaborn vs. Matplotlib

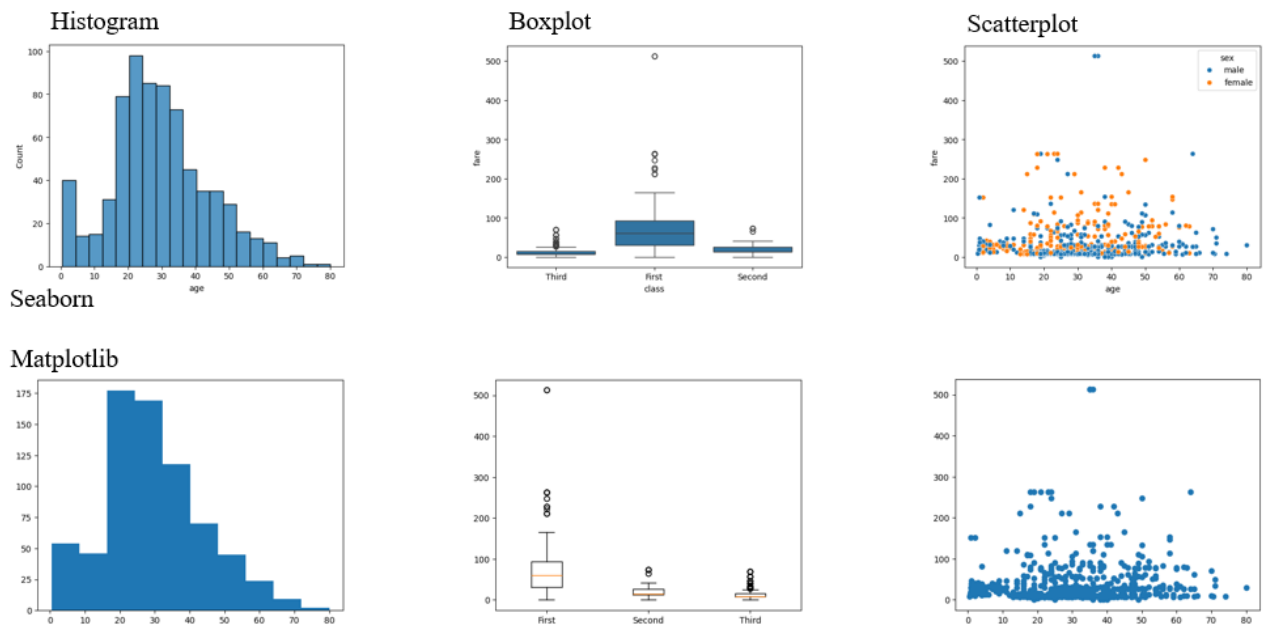


Tabelle 12: Übersicht aller nativen Plottypen³⁴

Plottyp	Seaborn (nativ)	Matplotlib (nativ)
Histogramm	✓ (histplot)	✓ (hist)
Scatterplot	✓ (scatterplot)	✓ (scatter)
Liniendiagramm	✓ (lineplot)	✓ (plot)
Boxplot	✓ (boxplot)	✓ (boxplot)
Violinplot	✓ (violinplot)	✓ (violinplot)
Swarmplot	✓ (swarmplot)	X
Stripplot	✓ (stripplot)	X
Countplot	✓ (countplot)	X
Barplot	✓ (barplot)	✓ (bar)
Pointplot	✓ (pointplot)	X
Heatmap	✓ (heatmap)	✓ (imshow, eingeschränkt)
Clustermap	✓ (clustermap)	X
Pairplot	✓ (pairplot)	X
Jointplot	✓ (jointplot)	X
KDE-Plot	✓ (kdeplot)	X
ECDF-Plot	✓ (ecdfplot)	X
Rugplot	✓ (rugplot)	X
Regressionsplot	✓ (regplot/lmplot)	X
Residualplot	✓ (residplot)	X
Boxenplot	✓ (boxenplot)	X
Polar diagramm	X	✓ (polar())
3D-Diagramme	X	✓ (mpl_toolkits.mplot3d)
Konturdiagramm	X	✓ (contour(), contourf())
Stammplot	X	✓ (stem())
Stufenplot	X	✓ (step())
Vektorfeld-Diagramm	X	✓ (quiver(), streamplot())
Fehlerbalken	X	✓ (errorbar())
Kreisdiagramm	X	✓ (pie())
Horizontales Balkendiagramm	X	✓ (barh())
Stackplot	X	✓ (stackplot())

³⁴ Vgl. API reference, o.J., o.S./ Vgl. Plot types, o.J., o.S.

Abbildung 4: Beispielhafte Plotvielfalt: Seaborn vs. Matplotlib

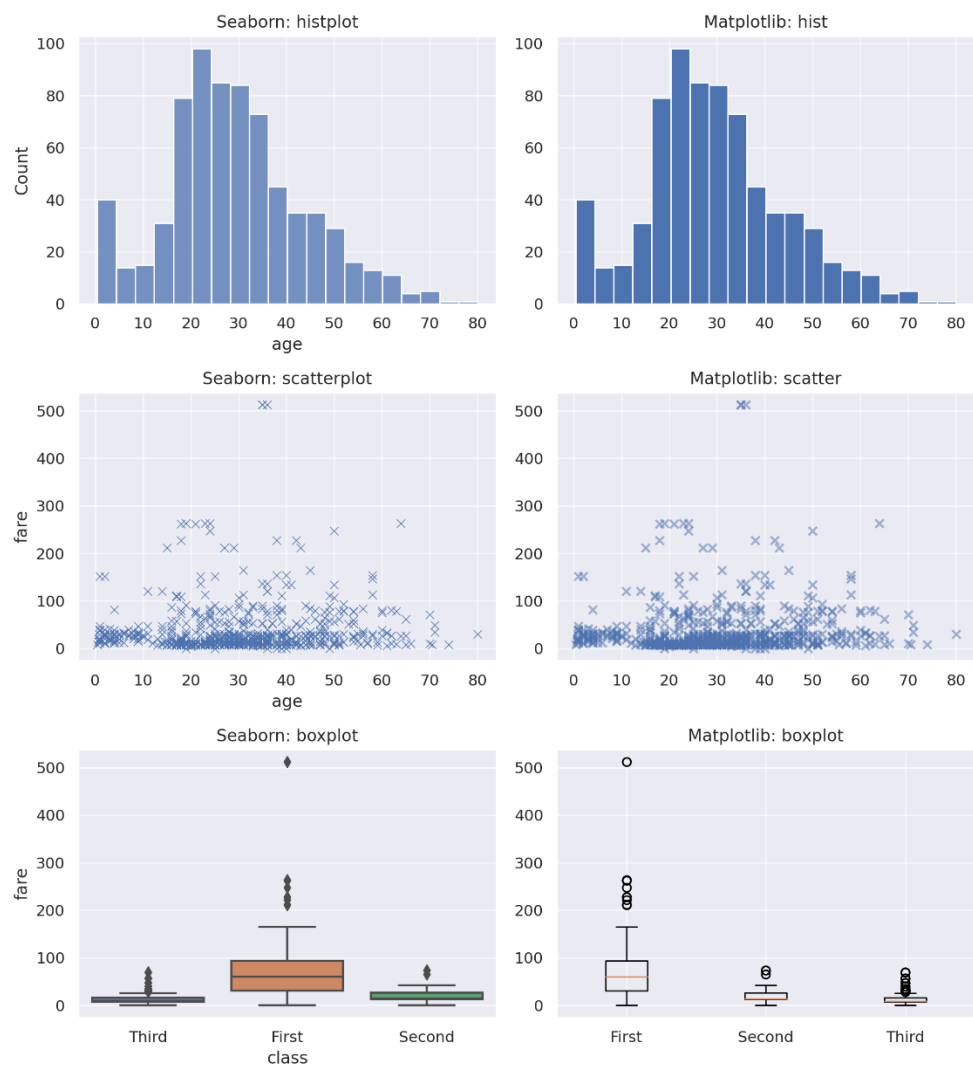


Abbildung 5: Unterschiedliche Anpassungslogik: Farbe

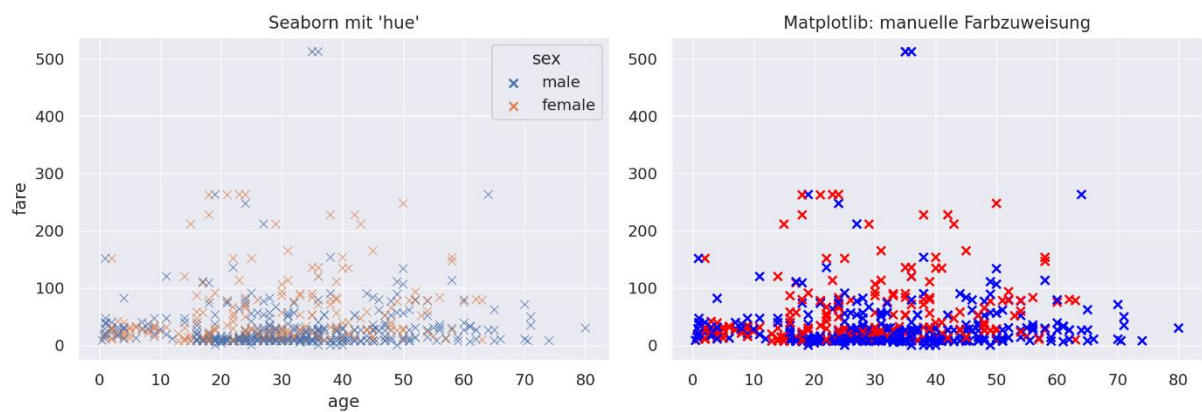


Abbildung 6: Seaborn: FacetGrid (age nach Geschlecht)

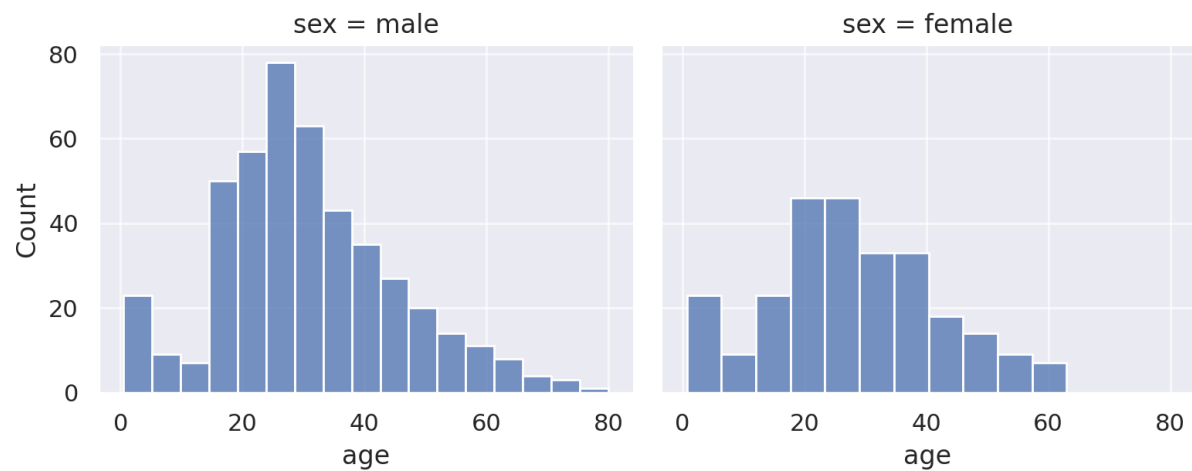
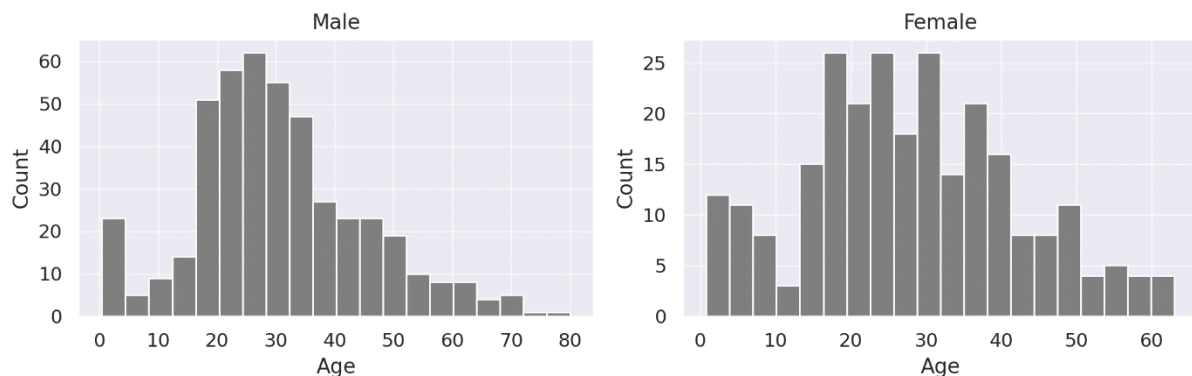


Abbildung 7: Matplotlib: Subplots (age nach Geschlecht)



Ehrenwörtliche Erklärung

„Hiermit bestätige ich, dass ich die vorliegende Arbeit persönlich und selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder sinngemäß anderen Quellen entnommen wurden, sind als solche kenntlich gemacht. Die Zeichnungen, Abbildungen und Tabellen in dieser Arbeit sind von mir selbst erstellt oder wurden mit einem entsprechenden Quellennachweis versehen. Diese Arbeit wurde weder in gleicher noch in ähnlicher Form von mir an anderen Hochschulen zur Erlangung eines akademischen Abschlusses eingereicht“.